

Tool: Camunda Modeler (BPMN) | draw.io / PlantUML (Sequence Diagram)

Detail soal:

- Soal Bagian A (3 soal) & B (2 soal) dikerjakan di Camunda Modeler (BPMN) atau draw.io/PlantUML (Sequence Diagram), kemudian hasil diagram di-export sebagai gambar (.png/.jpg) dan dilampirkan bersama dokumen jawaban.

BAGIAN A — STUDI KASUS BPMN

Petunjuk: Buat atau analisis diagram BPMN menggunakan Camunda Modeler. Export hasil sebagai .png dan lampirkan bersama dokumen jawaban.

SC-01 [Mid-Level] Membuat BPMN flow proses multi approval

SKENARIO: Proses Konfigurasi Multi-Approval dengan SLA & Request Clarification

Sebuah sistem perbankan memiliki fitur Konfigurasi Parameter Sistem yang melibatkan 3 role dalam alur multi-approval. Dibanding approval sederhana, skenario ini menambahkan batas waktu review (SLA) dan opsi Request Clarification dari Checker ke Maker.

Role & Tanggung Jawab:

Maker:

- Mengisi form konfigurasi yang memiliki **field wajib dan field opsional**.
- Sistem menjalankan **validasi 2 tahap** sebelum submission: (1) validasi mandatory field, (2) validasi business rule — misalnya nilai tidak boleh melebihi limit sistem.
- Jika validasi gagal, Maker harus memperbaiki sebelum bisa submit.
- Jika konfigurasi di-reject atau ada request clarification, Maker menerima notifikasi beserta keterangan alasan.

Checker:

- Menerima notifikasi saat ada konfigurasi masuk untuk direview.
- Memiliki **SLA review 2 hari kerja**. Jika melampaui SLA, sistem otomatis mengirim peringatan eskalasi ke Supervisor Checker.
- Checker bisa memilih 3 opsi:
 - **Approve** → lanjut ke Signer
 - **Reject** dengan keterangan → kembali ke Maker untuk revisi

- **Request Clarification** → Maker diminta memberikan penjelasan tambahan tanpa membuat konfigurasi baru, lalu kembali ke Checker untuk direview ulang

Signer:

- Menerima notifikasi hanya setelah Checker Approve.
- Memiliki SLA review 1 hari kerja. Jika melampaui SLA, sistem mengirimkan notifikasi reminder.
- Signer bisa memilih 2 opsi:
 - Approve Final → konfigurasi aktif di sistem
 - Reject Total → kembali ke Maker dari awal dengan keterangan alasan

Sistem (ReactJS / Laravel / Camunda 7):

- Menjalankan validasi 2 tahap sebelum submission.
- Memantau SLA setiap role menggunakan Timer Intermediate Event.
- Mengirim notifikasi ke Checker / Signer / Maker sesuai alur.
- Menyimpan status konfigurasi: Draft → Pending Checker → Pending Clarification → Pending Signer → Active / Rejected.

Tugas:

Buat BPMN diagram lengkap menggunakan Camunda Modeler dengan ketentuan berikut:

- Gunakan 4 swimlane: Maker | Checker | Signer | Sistem.
- Sertakan 5 XOR Gateway untuk setiap percabangan keputusan:
 - Validasi mandatory field lolos?
 - Validasi business rule lolos?
 - Keputusan Checker? (Approve / Reject / Request Clarification)
 - Clarification dijawab Maker? (Ya → kembali ke Checker / Tidak → dianggap Reject)
 - Keputusan Signer? (Approve Final / Reject Total)
- Sertakan Intermediate Event berikut:
 - Timer Intermediate Event: SLA Checker 2 hari kerja → trigger peringatan eskalasi ke Supervisor

• Timer Intermediate Event: SLA Signer 1 hari kerja → trigger peringatan eskalasi ke Direktur • Intermediate Message Event: notifikasi ke Checker setelah submit • Intermediate Message Event: notifikasi Request Clarification ke Maker • Intermediate Message Event: notifikasi ke Signer setelah Checker Approve d. Sertakan minimal 3 End Event yang berbeda: Konfigurasi Active, Konfigurasi Rejected by Checker, Konfigurasi Rejected by Signer. e. Label setiap task dan gateway harus deskriptif dan spesifik — hindari label generik seperti "Approve" saja, gunakan "Checker Approve Konfigurasi".

SC-02 [Mid-Level] Membuat BPMN flow proses restaurant order management	
SKENARIO: Manajemen Order Restoran — Alur Walk-In & Online (Common Case) Sebuah restoran modern menerima pesanan dari dua channel secara bersamaan: pelanggan yang datang langsung (walk-in) dan pelanggan yang memesan via aplikasi mobile (online). Kedua channel ini bermuara ke dapur yang sama, sehingga Chef harus mengelola antrian order dari dua sumber secara paralel.	
Aktor yang terlibat: • Pelanggan Walk-In: Pelanggan yang datang langsung ke restoran. • Pelanggan Online: Pelanggan yang memesan via aplikasi mobile. • Waitress: Melayani pelanggan walk-in, mencatat pesanan, dan menyajikan makanan. • Chef: Menerima dan memproses order dari dua channel (walk-in dan online) secara paralel sesuai antrian. • Kurir: Mengambil dan mengantarkan pesanan online ke alamat pelanggan. • Sistem (POS & Aplikasi Mobile): Mengelola input pesanan, pembayaran online, pengiriman order ke dapur, dan penugasan kurir.	
Alur Proses	
Walk-In	Online

1. Pelanggan walk-in tiba di restoran, duduk di meja, dan Waitress memberikan menu. 2. Pelanggan memilih menu dan menyampaikan pesanan kepada Waitress. 3. Waitress menginput pesanan ke Sistem POS → Sistem mencetak struk order di dapur. 4. [Percabangan] Chef mengecek ketersediaan bahan baku. Jika TIDAK TERSEDIA: Chef informasikan ke Waitress → Waitress tawarkan menu pengganti ke Pelanggan → Pelanggan pilih menu pengganti (kembali ke langkah 2), ATAU Pelanggan memutuskan membatalkan pesanan → End Event: Pesanan Walk-In Dibatalkan. 5. Jika bahan TERSEDIA: Chef memasak pesanan → Chef tandai pesanan selesai di Sistem POS → Waitress ambil dan sajikan makanan ke meja pelanggan. 6. Pelanggan minta bill → Waitress cetak bill → Pelanggan bayar (tunai atau kartu) → End Event: Transaksi Walk-In Selesai.	1. Pelanggan online membuka aplikasi, memilih menu, mengisi alamat pengiriman, dan memilih metode pembayaran (transfer, e-wallet, atau COD). 2. [Percabangan Pembayaran] Jika metode non-COD: Sistem memproses pembayaran. Jika GAGAL: Sistem tampilkan notifikasi gagal → Pelanggan coba ulang atau batalkan → End Event: Order Online Dibatalkan. Jika BERHASIL atau metode COD: Sistem kirim order otomatis ke dapur. 3. [Paralel Flow di Dapur] Chef menerima order dari walk-in DAN online secara bersamaan — proses memasak berjalan paralel sesuai antrian masuk. 4. Chef selesai memasak order online → Chef tandai selesai → Sistem otomatis menugaskan Kurir yang tersedia. 5. Kurir mengambil makanan di restoran → Kurir mengantarkan ke alamat pelanggan → Pelanggan mengkonfirmasi penerimaan di aplikasi → End Event: Order Online Selesai.
Tugas: a. Buat BPMN diagram lengkap menggunakan Camunda Modeler dengan ketentuan berikut: b. Gunakan minimal 5 swimlane: Pelanggan Walk-In Waitress Chef Pelanggan Online Kurir — tambahkan lane Sistem jika diperlukan. c. unakan Event-Based Gateway atau XOR Gateway di awal untuk memisahkan dua channel order (walk-in dan online) sebagai dua alur yang berjalan independen. d. Representasikan paralel flow di dapur menggunakan AND Gateway (Parallel Gateway) — Chef menerima order dari kedua channel secara bersamaan. e. Sertakan minimal 4 XOR Gateway untuk setiap percabangan keputusan: ketersediaan bahan baku, pilihan pelanggan walk-in, status pembayaran online, dan pilihan metode bayar. f. Sertakan minimal 4 End Event yang berbeda: Transaksi Walk-In Selesai, Pesanan Walk-In Dibatalkan, Order Online Selesai, Order Online Dibatalkan. g. Label setiap task dan gateway harus deskriptif dan spesifik — hindari label generik seperti "Cek" saja, gunakan "Chef cek ketersediaan bahan baku".	

SC-03 [Mid-Level] Sequence Diagram: Request Download Laporan dengan Async Flow & Retry Mechanism

SKENARIO: Proses Request Download Laporan

Di sebuah sistem perbankan, user dapat mengajukan Request Download laporan rekonsiliasi dalam format CSV atau XLSX. Proses ini bersifat asynchronous karena generate file bisa memakan waktu lama tergantung rentang data yang dipilih. Sistem juga dilengkapi mekanisme retry jika generate file gagal, dan notifikasi real-time ketika file sudah siap.

Komponen (Lifeline) yang terlibat:

- User (Actor)
- Frontend (ReactJS)
- Backend API (Laravel)
- Report Service (background job / worker)
- Database (PostgreSQL)
- Notification Service (email / in-app push)

Alur proses:

1. User memilih filter laporan (Product, Feature, Date Period, Format: CSV atau XLSX) dan klik tombol Request Download.
2. Frontend mengirim POST /api/report/download-request ke Backend API beserta parameter filter.
3. Backend API memvalidasi parameter filter: cek kelengkapan field wajib, validasi format tanggal, dan validasi rentang maksimum data (maksimal 90 hari).
4. [alt — Parameter Tidak Valid] Backend return 400 Bad Request beserta pesan error spesifik (mis: "Date range exceeds 90 days") → Frontend tampilkan pesan error ke User → alur selesai.
5. [alt — Parameter Valid] Backend INSERT download request ke Database dengan status PENDING, lalu kirim task ke Report Service secara ASYNCHRONOUS → Backend segera return 202 Accepted ke Frontend → Frontend tampilkan status "Processing..." ke User.
6. Report Service mengambil data laporan dari Database (SELECT query berdasarkan filter).
7. [alt — Data Ditemukan] Report Service generate file CSV/XLSX → [alt — Generate Berhasil] Report Service UPDATE status menjadi READY + simpan file_url di Database → Report Service kirim notifikasi ke Notification Service secara ASYNCHRONOUS.
8. [alt — Generate Gagal] Report Service UPDATE status menjadi FAILED + increment retry_count di Database. [opt — Retry Masih Tersedia (retry_count < 3)] Report Service melakukan retry generate file (kembali ke langkah 6). [opt — Retry Habis (retry_count >= 3)] Report Service UPDATE status menjadi PERMANENTLY_FAILED → Notification Service kirim notifikasi gagal ke User.

9. [alt — Data Tidak Ditemukan] Report Service UPDATE status menjadi EMPTY → Notification Service kirim notifikasi "No data found for selected filter" ke User.
10. [loop — Polling setiap 5 detik, maksimal 10x] Frontend melakukan polling GET /api/download-status/{requestId} ke Backend → Backend SELECT status dari Database → Backend return status terkini ke Frontend.
11. [opt — Status READY] Frontend tampilkan tombol Download → User klik Download → Frontend kirim GET /api/download/{requestId} ke Backend → Backend validasi request (cek ownership dan expiry URL) → Backend return signed URL ke Frontend → Frontend redirect User ke file download → File terunduh.
12. [opt — Status FAILED / PERMANENTLY_FAILED] Frontend tampilkan pesan error beserta opsi Request Ulang kepada User.

Tugas:

Buat Sequence Diagram lengkap menggunakan draw.io atau PlantUML dengan ketentuan berikut:

1. Tampilkan semua 6 lifeline: User | Frontend | Backend API | Report Service | Database | Notification Service.
2. Bedakan pesan synchronous (→ solid arrow) dan asynchronous (->> dashed open arrow) — terutama:
 - a. Trigger ke Report Service wajib notasi asynchronous
 - b. Notifikasi dari Report Service ke Notification Service wajib notasi asynchronous
3. Gunakan alt fragment untuk kondisi berikut:
 - a. Parameter tidak valid vs valid
 - b. Data ditemukan vs tidak ditemukan
 - c. Generate file berhasil vs gagal
4. Gunakan opt fragment untuk:
 - a. Retry generate file (retry_count < 3)
 - b. Status READY → tampilkan tombol download
 - c. Status FAILED → tampilkan pesan error + opsi request ulang
5. Gunakan loop fragment untuk proses polling setiap 5 detik maksimal 10x.
6. Semua return message wajib digambarkan dengan dashed arrow (- - →), termasuk:
 - a. Return 202 Accepted dari Backend ke Frontend
 - b. Return status dari polling
 - c. Return signed URL saat download

7. Bedakan 202 Accepted (response async — proses belum selesai) dengan 200 OK (response sync — data langsung dikembalikan) pada setiap response Backend.

SC-04 [Mid-Level] Sequence Diagram: Transfer Dana via Mobile Banking (BI-FAST) — Async Callback + Retry + Paralel Notifikasi

SKENARIO: Transfer Dana Antar Bank via BI-FAST dengan Retry Mechanism & Notifikasi Real-Time ke Dua Pihak

Nasabah melakukan transfer dana ke rekening bank lain melalui Mobile Banking. Proses ini melibatkan validasi bertingkat, pengiriman instruksi ke jaringan BI-FAST secara asynchronous, mekanisme retry jika BI-FAST tidak merespons, dan notifikasi real-time ke dua pihak (pengirim dan penerima) yang berjalan secara paralel.

Komponen (Lifeline) yang terlibat:

- Nasabah (Actor)
- Mobile Banking App (Frontend)
- Backend API (Laravel)
- Core Banking System
- Jaringan BI-FAST (Bank Indonesia)
- Notification Service

Alur proses:

Alur 1 — Inquiry Rekening Tujuan:

1. Nasabah membuka fitur Transfer, mengisi nomor rekening tujuan, nominal, dan keterangan, lalu klik Lanjut.
2. Mobile App mengirim GET /api/inquiry/account?account_no=&bank_code= ke Backend (synchronous).
3. Backend meneruskan inquiry ke BI-FAST (POST /biffast/inquiry) — synchronous, menunggu respons.
4. [alt — Rekening tidak ditemukan] BI-FAST return ACCOUNT_NOT_FOUND → Backend return 404 Not Found → Mobile App tampilkan error "Rekening tujuan tidak ditemukan" → selesai.
5. [alt — Rekening ditemukan] BI-FAST return nama pemilik rekening → Backend return 200 OK + nama pemilik → Mobile App tampilkan nama pemilik untuk konfirmasi Nasabah.

Alur 2 — Validasi Transaksi:

6. Nasabah mengkonfirmasi nama pemilik dan memasukkan PIN transaksi.
7. Mobile App mengirim POST /api/transfer ke Backend (synchronous) dengan payload: rekening tujuan, nominal, PIN terenkripsi.
8. Backend melakukan 2 validasi terpisah ke Core Banking secara sequential:
 - a. POST /cbs/verify-pin — verifikasi PIN Nasabah
 - b. POST /cbs/check-balance — cek kecukupan saldo + limit transaksi harian.

9. [alt — PIN salah] Core Banking return INVALID_PIN → Backend return 401 Unauthorized → Mobile App tampilkan error "PIN tidak sesuai" → selesai.
10. [alt — Saldo tidak cukup / limit terlampaui] Core Banking return INSUFFICIENT_BALANCE atau LIMIT_EXCEEDED → Backend return 422 Unprocessable Entity + pesan spesifik → Mobile App tampilkan error → selesai.
11. [alt — Validasi berhasil] Core Banking melakukan DEBIT rekening Nasabah (hold dana) → Core Banking return konfirmasi debit + transaction_id → Backend UPDATE status transaksi ke PROCESSING di Core Banking.

Alur 3 — Pengiriman ke BI-FAST (Async):

12. Backend mengirim instruksi transfer ke BI-FAST (POST /biffast/transfer) secara ASYNCHRONOUS — tidak menunggu respons.
13. Backend segera return 202 Accepted + transaction_id ke Mobile App.
14. Mobile App tampilkan status "Transfer Sedang Diproses" + transaction_id ke Nasabah.

Alur 4 — Callback & Retry Mechanism:

15. [opt — BI-FAST tidak mengirim callback dalam 30 detik] Backend melakukan 1x retry — kirim ulang instruksi transfer ke BI-FAST dengan request yang sama + flag is_retry=true.
 - a. Jika setelah retry BI-FAST masih tidak merespons dalam 30 detik tambahan: Backend lanjut ke alur FAILED (reverse dana).
16. BI-FAST mengirim callback/webhook ke Backend secara ASYNCHRONOUS dengan status hasil transfer.
17. [alt — Callback: SUCCESS]
 - a. Backend UPDATE status transaksi menjadi SUCCESS di Core Banking
 - b. Backend trigger Notification Service secara ASYNCHRONOUS
 - c. [par] Notification Service mengirim notifikasi secara PARALEL:
 - i. Kirim push notification + SMS ke Nasabah Pengirim: "Transfer Rp X ke [Nama Penerima] berhasil. No. Ref: [ref_id]"
 - ii. Kirim push notification ke Nasabah Penerima via bank tujuan: "Dana masuk Rp X dari [Nama Pengirim]"
 - d. Mobile App menampilkan "Transfer Berhasil" + nomor referensi ke Nasabah.
18. [alt — Callback: FAILED / Timeout setelah retry]
 - a. Backend instruksikan Core Banking untuk melakukan REVERSE — kembalikan dana yang di-hold ke rekening Nasabah.
 - b. Core Banking UPDATE status transaksi menjadi REVERSED.
 - c. Backend trigger Notification Service secara ASYNCHRONOUS.
 - d. Notification Service kirim push notification + SMS ke Nasabah Pengirim: "Transfer Rp X gagal. Dana telah dikembalikan ke rekening Anda."
 - e. Mobile App menampilkan "Transfer Gagal — Dana Dikembalikan" ke Nasabah.

Tugas:

Buat Sequence Diagram lengkap menggunakan draw.io atau PlantUML dengan ketentuan berikut:

- a. Tampilkan semua 6 lifeline: Nasabah | Mobile Banking App | Backend API | Core Banking System | Jaringan BI-FAST | Notification Service.
- b. Bedakan notasi synchronous (→) dan asynchronous (->>) — instruksi transfer ke BI-FAST, callback dari BI-FAST, dan trigger ke Notification Service WAJIB notasi asynchronous.
- c. Semua return message wajib digambarkan dengan dashed arrow (-->).
- d. Gunakan alt fragment untuk kondisi berikut:
 - a. Inquiry: rekening tidak ditemukan vs ditemukan
 - b. Validasi: PIN salah vs saldo tidak cukup vs validasi berhasil
 - c. Callback: SUCCESS vs FAILED/Timeout
- e. Gunakan opt fragment untuk retry mechanism — BI-FAST tidak merespons dalam 30 detik → Backend kirim 1x retry.
- f. Gunakan par fragment untuk notifikasi paralel ke Nasabah Pengirim dan Penerima saat transfer SUCCESS.
- g. Validasi PIN dan cek saldo ke Core Banking harus digambarkan sebagai 2 call terpisah secara sequential, bukan 1 call.
- h. Label setiap pesan menggunakan nama endpoint atau deskripsi aksi spesifik — sertakan response code (404, 401, 422, 202, 200) pada setiap return message.

